

RubikSim — 3D Rubik's Cube Simulation & Synthetic Vision Environment

Project Overview

RubikSim is a high-performance 3D Rubik's Cube simulation environment built in C++ for real-time interaction, visualization, procedural scene generation, and synthetic dataset creation for computer vision and embodied AI research.

The project combines:

- real-time graphics
- simulation architecture
- spatial reasoning systems
- procedural data generation
- computer vision tooling

The application supports interactive Rubik's cube manipulation across multiple cube sizes (2x2, 3x3, 4x4, NxN), while also functioning as a synthetic rendering environment capable of generating labeled datasets for AI perception tasks.

Core Goals

Primary Goals

1. Learn modern C++ systems architecture
 2. Learn real-time graphics programming
 3. Learn 3D math and transforms
 4. Build simulation-engine style architecture
 5. Learn synthetic data generation pipelines
 6. Build portfolio signal for:
 7. graphics
 8. simulation
 9. robotics
 10. embodied AI
 11. vision systems
-

High-Level Features

Interactive Cube Simulation

- Interactive 3D Rubik's cubes
 - Support:
 - 2x2
 - 3x3
 - 4x4
 - arbitrary NxN
 - Mouse interaction
 - Keyboard controls
 - Camera orbit/pan/zoom
 - Smooth animated turns
 - Scramble generator
 - Solve reset
 - Timer system
-

Synthetic Vision Environment

- Automated scene randomization
 - Dataset generation pipeline
 - Multi-camera rendering
 - Ground-truth export
 - Domain randomization
 - Image export
-

Technical Stack

Language

- C++20
-

Graphics API

Recommended

- OpenGL 4.6

Alternative

- Vulkan (harder, stronger systems signal)
-

Libraries

Purpose	Library
Window/Input	GLFW
OpenGL Loader	GLAD
Math	GLM
UI	Dear ImGui
Image Export	stb_image_write
Model Loading	Assimp (optional)
Physics	Bullet Physics (optional)

System Architecture

Core Engine Systems

1. Renderer

Responsible for:

- rendering meshes
- shaders
- camera matrices
- framebuffers
- lighting
- post-processing

Responsibilities

- GPU buffer management
 - shader compilation
 - render loop
 - shadow maps
 - offscreen rendering
-

2. Scene System

Tracks:

- objects
- transforms
- cameras
- lights

Entities

```
Entity
├─ Transform
├─ MeshRenderer
├─ Camera
├─ Light
```

3. Cube System

Handles:

- cube state
- rotations
- indexing
- move validation
- scramble generation

Features

- NxN cube generation
- procedural cubie placement
- face indexing
- move notation parsing

Example Moves

```
R
U
F'
L2
Rw
```

4. Animation System

Handles:

- smooth face rotations
- interpolation
- easing
- queued move execution

Concepts

- quaternion interpolation
 - rotation matrices
 - transform hierarchies
-

5. Camera System

Features

- free orbit camera
- zoom
- pan
- cinematic camera mode
- scripted camera paths

Math Concepts

- view matrices
 - projection matrices
 - Euler vs quaternions
-

6. Input System

Features

- keyboard input
 - mouse dragging
 - raycasting
 - cubie selection
-

7. Dataset Generator

The most important system for AI relevance.

Responsibilities

- procedural scene generation
 - randomized rendering
 - image export
 - metadata export
-

Rendering Features

Lighting

- directional lights
 - point lights
 - HDR environment lighting
-

Materials

- glossy plastic cube materials
 - roughness variation
 - metallic workflow
-

Shadows

- shadow mapping
 - soft shadows (optional)
-

Post Processing

Optional:

- bloom
 - FXAA
 - tone mapping
 - motion blur
-

Synthetic Dataset Generation

Main AI-Oriented Feature

The engine can procedurally generate labeled datasets.

Generation Pipeline

Step 1 — Generate Cube State

Random:

- solved
 - partially solved
 - fully scrambled
-

Step 2 — Randomize Environment

Randomize:

- camera position
- FOV
- lighting direction
- light intensity
- background
- cube colors
- material properties

This is called:

Domain Randomization

A major robotics/vision technique.

Step 3 — Render Outputs

RGB Image

rgb_0001.png

Segmentation Mask

mask_0001.png

Depth Map

depth_0001.png

Normal Map

normal_0001.png

Step 4 — Export Metadata

```
{
  "cube_size": 3,
  "cube_state": "...",
  "camera_position": [1.2, 0.5, -3.1],
  "camera_rotation": [ ... ],
  "lighting": {
    "intensity": 2.3
  }
}
```

Machine Learning Extensions

Extension 1 — Cube State Detection

Train a CNN to:

- infer cube state from image
-

Extension 2 — Pose Estimation

Train a model to:

- estimate camera orientation
 - estimate cube orientation
-

Extension 3 — Cube Solver Agent

Train:

- RL agent
- search agent

To solve cubes.

Extension 4 — Multi-View Reconstruction

Generate:

- multiple camera views

Then reconstruct:

- cube geometry
 - scene understanding
-

UI Features

Built using Dear ImGui.

Panels

Cube Controls

- cube size
 - scramble
 - reset
 - solve
 - animation speed
-

Dataset Controls

- dataset size
 - output directory
 - image resolution
 - randomization toggles
-

Rendering Controls

- shadows
 - bloom
 - lighting
 - camera settings
-

Statistics

- FPS
 - frame time
 - render time
 - GPU memory
-

Interaction Features

Mouse Picking

Click cube faces directly.

Requires:

- raycasting
- coordinate conversion

Important graphics skill.

Drag Rotations

Drag:

- face rotations
 - camera movement
-

Advanced Features

Phase 2 Features

1. GPU Instancing

Efficient rendering of many cubies.

2. Compute Shader Dataset Generation

Generate datasets entirely on GPU.

3. ECS Architecture

Refactor into:

- Entity Component System

Good systems-engineering signal.

4. Physics Integration

Use Bullet Physics for:

- rigid body motion
 - collisions
-

5. Robotic Manipulation Mode

Add:

- robot arm
- grasp mechanics

This strongly increases embodied AI relevance.

File Structure

```
RubikSim/
|
|— assets/
|— shaders/
|— datasets/
|— src/
|   |— renderer/
|   |— scene/
|   |— cube/
|   |— animation/
|   |— dataset/
|   |— ui/
|   └─ math/
|
|— third_party/
|— CMakeLists.txt
└─ README.md
```

Important Graphics Concepts You Will Learn

Concept	Importance
Transformation matrices	Critical
Projection matrices	Critical
Camera systems	Critical
GPU buffers	Critical
Shader programming	Critical
Framebuffers	Important
Raycasting	Important
Quaternions	Critical
Lighting models	Important
Scene graphs	Important

Important C++ Concepts You Will Learn

Concept	Importance
RAII	Critical
Smart pointers	Critical
Templates	Moderate
ECS patterns	Important
Resource management	Critical
Multithreading	Optional
Memory layout	Important

Resume Signal

This project would signal:

Area	Signal Strength
C++	Strong
Graphics	Strong
Simulation	Strong
Robotics	Moderate
Embodied AI	Moderate
Vision Systems	Moderate
ML Systems	Moderate

If extended with:

- physics
- robotics
- dataset generation
- learning components

then signal becomes much stronger.

Difficulty Assessment

Base Version

Intermediate

Full Vision/Data Version

Advanced

Vulkan Version

Very Advanced

Recommended Development Timeline

Phase 1 — Graphics Foundations

1–2 weeks

Build:

- window
 - renderer
 - camera
 - cube rendering
-

Phase 2 — Cube Logic

1 week

Build:

- cube state system
 - rotations
 - animations
-

Phase 3 — Interaction

1 week

Build:

- mouse controls
 - picking
 - UI
-

Phase 4 — Dataset System

1–2 weeks

Build:

- export pipeline
 - randomization
 - metadata generation
-

Phase 5 — Advanced Rendering

Optional

Add:

- shadows
 - PBR
 - post-processing
-

Final Resume Description Example

Developed a configurable 3D Rubik's cube simulation environment in C++ and OpenGL supporting procedural NxN cube generation, real-time interaction, domain-randomized rendering, and synthetic dataset generation for computer vision research. Implemented custom rendering, animation, camera, and scene systems with automated export of RGB, depth, segmentation, and metadata annotations.